

Bindings	<code>pryr</code>
Set	Frame
Environment	NOTE:
NOTE:	<code>new.env()</code>
<code>ls()</code>	<code>assign()</code>

Package that provides tools to peek inside R to see what it's doing	Name-value pairs
• A data structure that is a set of bindings • NOTE: You can't create frame in R	An unordered collection of unique elements
Know how the diagrams of environments work	• A frame and a pointer to another environment called its enclosing/parent environment • The recursive definition terminates at an empty frame called the empty environment
Creates a new environment enclosed by the environment in which it is called	The set of all environments form a directed rooted tree
• Creates bindings in an environment • <code>assign() ≈ <-</code>	Lists the bindings in the current or given environment

<code>get()</code>	<code>rm()</code>
NOTE:	Environments operate like:
<code>list2env()</code>	Environments and Memory
Multiple Bindings	Lexical Scoping
<code>parent.env()</code>	What are the three System Environments?

Removes bindings	• Accesses bindings • <code>get() ≈ []</code>
Named lists	<code>[]</code> and <code>\$</code> are defined for environments
Environments have efficient memory allocation • Duplicated bindings aren't duplicated in memory • Modifications make copies	Turns a list into an environment
First seeks binding in the environment requested, and then goes to the enclosing/parent environment	• The same value can be bound to the same name in different environments • Objects can have different binding environments
1. <code>globalenv()</code> 2. <code>baseenv()</code> 3. <code>emptyenv()</code>	The enclosing/parent environment

`globalenv()`

`baseenv()`

`emptyenv()`

`environment()`

`search()`

`libPaths()`

`attach()`

`detach()`

Why using `attach()` is a bad idea

`pryr::where()`

• Environment of the package <code>base</code> • Its parent is the empty environment	• The global environment • The default workspace • Its parent is the package you most recently attached
• The current environment; callable inside functions • <code>environment(f)</code> = enclosing/parent environment of functions	• The empty environment ie environment "Adam" • It parents other environments but has no bindings itself
Gives you paths to where are your packages are stored	Lists the parent chain of <code>.GlobalEnv</code>
• Removes environments from the path • Can also be used to un-library a package	• A way to put data structures onto the path • Does NOT create new variables in the Global Environment
• Returns the environment of a binding • Recursively moves up the environment tree	• It makes copies of things, and it is easy to forget to <code>detach()</code> • You never need to use it if you understand scoping • It is considered a noob move in the R community

Namespaces	Unexported Functions
<code>utils::apropos()</code> and <code>utils::find()</code>	Formals of a Function
Calling Arguments of a Function	Four Environments associated with Functions
Enclosing Environment	Binding Environment(s)
Execution Environment	Calling Environment

• Functions in a package that are not made available to the user by default • Can be accessed using the <code>:::</code> operator	Special environments associated with packages
The arguments that a function is defined with	Find all the names in <code>search()</code> matching a regular expression
1. The enclosing environment 2. The binding environment(s) 3. The execution environment 4. The calling environment	The arguments that a function is called/invoked with
• Environments that have bindings to the function • EXAMPLES: Package, Namespace, and Imports	Where the function is defined; used for lexical scoping
Where the function is used; used for dynamic scoping	• Ephemeral environment created to host function execution • Enclosed by enclosing environment of function